

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

9. Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust	4
9.1. Security Challenges trong Microservices	5
9.1.1. Vấn đề: attack surface mở rộng	5
9.1.2. Nguyên tắc “Defense in Depth”	6
9.1.3. Advanced: mTLS, Secrets Management, OAuth2 Scopes	7
9.2. JWT — Cấu trúc và Cơ chế	8
9.2.1. Vấn đề: sessions không scale trong microservices	8
9.2.2. JWT (JSON Web Token)	8
9.2.3. JWT Flow trong KBM	9
9.3. Chiến lược Xác thực Kép	12
9.3.1. Vấn đề: thực hiện xác thực ở đâu? Mỗi dịch vụ hay chỉ tại gateway?	12
9.3.2. Cách tiếp cận của LMS: Xác thực Kép	12
9.4. OAuth2 — External Identity Provider	14
9.4.1. OAuth2 Authorization Code Flow	14
9.4.2. Multiple Authentication Methods	15
9.4.3. Service-to-Service: Client Credentials Flow	15
9.5. RBAC — Role-Based Access Control	15
9.5.1. Vấn đề: ai được làm gì?	16
9.5.2. RBAC Implementation trong KBM	16
9.5.3. API-driven Route Protection (Frontend)	17
9.6. Case Study: KBM	17
9.6.1. Phân tích tổng hợp	18
9.6.2. Đề xuất migration	19
9.6.3. Secret Management trong Production	20
9.7. Cô lập thực thi và Lockdown thi cử — Hai bài học phòng thủ	23
9.7.1. Chạy mã sinh viên: rủi ro “no sandbox” và lời giải cô lập	23
9.7.2. Lockdown thi cử và defense-in-depth chống gian lận	24
9.8. Tổng kết	24
9.9. Đọc thêm	25
Tài liệu tham khảo	25

9.

CHƯƠNG 9.

Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust

“Security is not an afterthought. In a distributed system, every service is a potential attack surface.”

— Sam Newman, *Building Microservices* [4a]

MỤC TIÊU HỌC TẬP

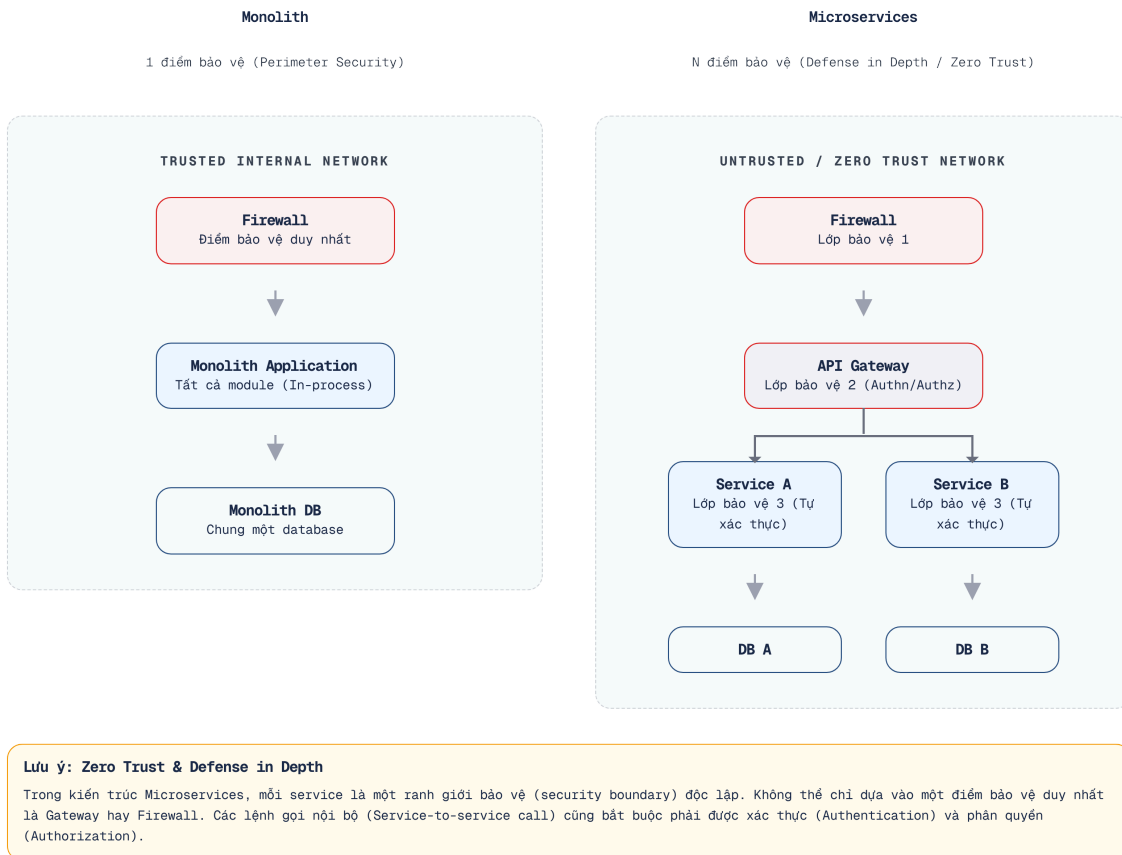
- Hiểu các thách thức bảo mật đặc thù của kiến trúc microservices
- Nắm vững cấu trúc JWT (JSON Web Token): header, payload, signature
- Phân biệt hai chiến lược xác thực: DB-based (xác thực đầy đủ) vs claims-only (stateless)
- Hiểu OAuth2/OIDC (OpenID Connect) flow và tích hợp external identity provider (Microsoft/Google)
- Thiết kế RBAC (Role-Based Access Control) cho hệ thống đa vai trò
- Phân tích kiến trúc bảo mật của KBM

9.1. Security Challenges trong Microservices

Việc đảm bảo an ninh cho một hệ thống nguyên khối giống như bảo vệ một pháo đài duy nhất, nhưng với microservices, bài toán này biến thành việc phải bảo vệ từng tòa nhà, từng giảng đường nằm rải rác khắp khuôn viên đại học.

9.1.1. Vấn đề: attack surface mở rộng

Trong kiến trúc nguyên khối (monolith), bảo mật tập trung tại một điểm: một ứng dụng, một cơ sở dữ liệu, một tầng xác thực (authentication layer). Khi chuyển sang microservices, bề mặt tấn công (attack surface) mở rộng tỷ lệ thuận với số lượng dịch vụ:



Hình 9.1: Monolith (1 điểm bảo vệ) vs Microservices (N điểm bảo vệ)

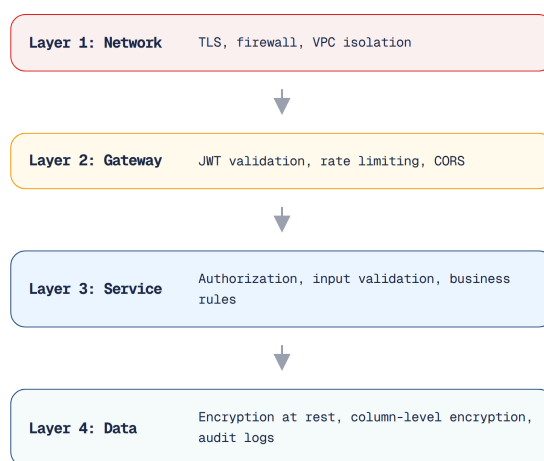
Sam Newman trong cuốn **Building Microservices** đã liệt kê ra năm thách thức bảo mật đặc thù khi chuyển dịch sang hệ thống phân tán, nơi sự tin tưởng mặc định không còn tồn tại và mỗi thành phần đều phải tự thiết lập cơ chế bảo vệ:

#	Thách thức	Monolith	Microservices
1	Authentication	Một session store	Mỗi dịch vụ cần xác thực danh tính — người truy cập là ai?
2	Authorization	Kiểm tra cục bộ (in-process)	Mỗi dịch vụ cần phải kiểm tra quyền truy cập — người dùng được phép thực hiện những hành động nào?
3	Service-to-service trust	Không có (in-process)	Dịch vụ A gọi dịch vụ B — làm thế nào B xác thực được A là đáng tin cậy?
4	Data in transit	Internal (in-process)	Network calls — cần encryption (TLS)
5	Secret management	Một config file	N dịch vụ kết hợp với M thông tin xác thực (secrets) = bài toán quản lý phức tạp

Bảng 9.1: Năm thách thức bảo mật đặc thù khi chuyển từ monolith sang microservices

9.1.2. Nguyên tắc “Defense in Depth”

Để đối phó với những thách thức phức tạp đa chiều, bảo mật trong môi trường phân tán bắt buộc phải dựa trên triết lý **defense in depth** (phòng thủ chiều sâu). Thay vì chỉ phụ thuộc vào một tường lửa duy nhất ở biên mạng, chúng ta phải xây dựng nhiều rào chắn bảo vệ nối tiếp nhau từ tầng mạng cho đến tận tầng dữ liệu:



Hình 9.2: Defense in Depth — bảo vệ nhiều lớp từ network đến data

Trong thực tế, mức độ áp dụng tùy thuộc vào **trust boundary**. Nếu mọi dịch vụ chạy trong mạng riêng (private network, VPC) và chỉ cổng giao tiếp (gateway) công khai ra Internet, ranh giới tin cậy (trust boundary) nằm tại cổng giao tiếp (gateway) — các dịch vụ nội bộ có thể tin tưởng lẫn nhau ở một mức độ nhất định.

Zero Trust vs Pragmatic Trust

Nguyên lý “Zero Trust” (mọi yêu cầu đều phải được xác thực, mọi dịch vụ đều phải bị nghi ngờ) là một mô hình lý tưởng. Trong thực tế, các đội ngũ nhỏ thường áp dụng mô hình tin cậy thực dụng (**pragmatic trust**): cổng giao tiếp (gateway) chịu trách nhiệm xác thực danh tính, và các dịch vụ bên trong sẽ tin tưởng kết quả từ gateway. Newman trong [4a, Ch.9] ghi nhận rằng: mức độ tin cậy phù hợp phụ thuộc vào hồ sơ rủi ro (risk profile) — hệ thống tài chính đòi hỏi zero trust, nhưng hệ thống học thuật có thể áp dụng pragmatic trust. KBM là một hệ thống LMS học thuật, do đó việc áp dụng pragmatic trust là hoàn toàn hợp lý nếu ranh giới mạng nội bộ được kiểm soát chặt chẽ.

9.1.3. Advanced: mTLS, Secrets Management, OAuth2 Scopes

Khi hệ thống mở rộng quy mô (scale) hoặc mức độ rủi ro (risk profile) tăng lên (ví dụ: bổ sung phân hệ thanh toán, xử lý dữ liệu cá nhân), cần nâng cấp các biện pháp bảo mật:

mTLS (Mutual TLS) — Trong TLS thông thường, chỉ có máy khách xác thực máy chủ (ví dụ trình duyệt xác thực chứng chỉ HTTPS). Trong mTLS, **cả hai bên đều phải xác thực lẫn nhau**: Khi Dịch vụ A gọi Dịch vụ B, B sẽ kiểm tra chứng chỉ của A và ngược lại, A cũng kiểm tra chứng chỉ của B. Cơ chế này giúp ngăn chặn các dịch vụ lạ truy cập trái phép vào API nội bộ.

Khác với giao tiếp nội bộ thông thường qua HTTP không mã hóa tiềm ẩn nhiều rủi ro, mTLS thiết lập kênh truyền tải an toàn bằng cơ chế xác thực hai chiều nghiêm ngặt. Hệ quả là, mTLS ngăn chặn attacker khai thác các service nội bộ. Tuy nhiên, nhược điểm của nó là sự phức tạp trong quá trình triển khai, đòi hỏi các công cụ quản lý chứng chỉ tự động chuyên dụng như cert-manager hoặc Istio.

Trong KBM, mTLS hiện chưa phải ưu tiên đầu tiên nếu các services chỉ giao tiếp trong boundary nội bộ được kiểm soát. Tuy nhiên, khi triển khai phân tán ở quy mô lớn, mTLS giúp ngăn chặn các luồng tấn công lan truyền (lateral movement) trong trường hợp một điểm nút (node) bị xâm nhập.

Secrets Management — Credentials (DB passwords, API keys, JWT secrets) không nên nằm trong code hoặc Docker images:

Trong việc quản lý thông tin nhạy cảm, việc hardcode trực tiếp các bí mật (như `password: "abc123"`) vào mã nguồn là hành vi nguy hiểm nhất, dẫn đến nguy cơ lộ lọt dữ liệu vĩnh viễn qua lịch sử Git. Cấp độ bảo mật trung bình là sử dụng biến môi trường (environment variables) qua file `.env`, phương án này tách bạch mã nguồn khỏi cấu hình nhưng vẫn dễ bị đánh cắp nếu kẻ gian rà quét tiến trình chạy container. Giải pháp tiêu chuẩn cho hệ thống production là sử dụng các hệ thống quản lý bí mật chuyên dụng (Secrets Manager) như HashiCorp Vault, cung cấp cơ chế mã hóa tĩnh, tự động xoay vòng khóa và kiểm toán truy cập.

KBM hiện đang sử dụng các biến môi trường (environment variables) ở một số môi trường triển khai (đáp ứng đủ quy mô của phạm vi học thuật) — tuy nhiên, khóa bảo mật JWT cần được luân phiên (rotate) định kỳ và tuyệt đối không lưu trữ (commit) trực tiếp vào mã nguồn.

OAuth2 Scopes — Hiện tại KBM RBAC dùng roles (STUDENT, LECTURER, ADMIN). OAuth2 scopes mở rộng: thay vì “user có role ADMIN”, phạm vi quyền (scope) cho phép chỉ định “ứng dụng máy khách X có quyền đọc bài nộp nhưng không có quyền ghi điểm”. Cơ chế này đặc biệt phù hợp khi KBM cung cấp API cho các bên thứ ba (như ứng dụng di động độc lập hoặc tích hợp với các hệ thống khác).

9.2. JWT – Cấu trúc và Cơ chế

Khi số lượng sinh viên truy cập đồng thời vào hệ thống thư viện tăng vọt, việc duy trì một sổ điểm danh tập trung sẽ nhanh chóng bộc lộ nhiều điểm yếu. Khác với mô hình cũ, hệ thống cần một phương thức kiểm tra thẻ độc lập, phi tập trung hơn để phân tán áp lực xác thực.

9.2.1. Vấn đề: sessions không scale trong microservices

Trong kiến trúc nguyên khối, quá trình xác thực thường sử dụng cơ chế phiên làm việc phía máy chủ (**server-side session**): Người dùng đăng nhập, máy chủ tạo một phiên làm việc, lưu trữ trong bộ nhớ hoặc Redis, sau đó trả về mã định danh phiên (session ID) dưới dạng cookie. Các yêu cầu tiếp theo sẽ gửi kèm cookie này để máy chủ tra cứu và nhận diện người dùng.

Trong kiến trúc microservices, kho lưu trữ phiên (session store) tạo ra một điểm phụ thuộc tập trung (**centralized dependency**): việc mọi dịch vụ phải truy vấn cùng một cơ sở dữ liệu phiên sẽ gây ra hiện tượng thắt cổ chai và tạo thành điểm nghẽn duy nhất (single point of failure). Ngược lại, nếu mỗi dịch vụ có một kho lưu trữ phiên riêng, người dùng sẽ phải đăng nhập lại mỗi khi chuyển đổi giữa các dịch vụ.

9.2.2. JWT (JSON Web Token)

💡 Thi Giấy vs Thi Máy Tính

Session (Thi giấy) – Giám thị phải mang theo một danh sách điểm danh. Mỗi khi có sinh viên nộp bài, giám thị phải tra đúng dòng, đúng tên trong danh sách (Lookup session store). Nếu giám thị chuyển danh sách đi phòng khác, hệ thống sẽ rối loạn.

JWT (Thị máy tính) – Giống như một **Mã QR định danh** do Hệ thống Khảo thí cấp khi đăng nhập. Sinh viên cầm mã QR này nộp bài. Hệ thống chấm bài (Service) chỉ cần quét xem mã QR có chữ ký điện tử hợp lệ của phòng Khảo thí không, tự giải mã được mã SV mà không cần truy vấn vào cơ sở dữ liệu.

JWT giải quyết bằng cách mã hóa thông tin định danh (identity information) vào *chính token* – phi trạng thái (stateless), không cần kho lưu trữ phiên (session store) [4a, Ch.9]:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.      ← Header
eyJ1c2VySWQiOiJ1c2VyLTEyMyIsInR5cCI6IkpXVCJ9. ← Payload
VREVOVF9BRE1JTlTiImV4cCI6IjE2MTEyMDEwMDAwMDAwMH0.
SflKxwRJSMeKKF2QT4fwpMeJf33P0k6yJV adQssw5c ← Signature
```

Ba phần của JWT:



```
header.payload.signature
```

[Idea] Lưu ý

JWT chỉ được ký (signed), không mã hóa. Ai cũng có thể decode payload – không lưu thông tin nhạy cảm

Hình 9.3: Cấu trúc JWT – Header, Payload, Signature

Về mặt cấu trúc, một JWT hoàn chỉnh được ghép lại từ ba mảnh ghép mã hóa Base64URL riêng biệt. Phần đầu tiên là **Header** chứa thông tin về thuật toán mã hóa và loại token (ví dụ: `{"alg": "HS256"}`). Phần thứ hai là **Payload**, mang theo các dữ liệu định danh (claims) đặc biệt quan trọng của người dùng như `userId`

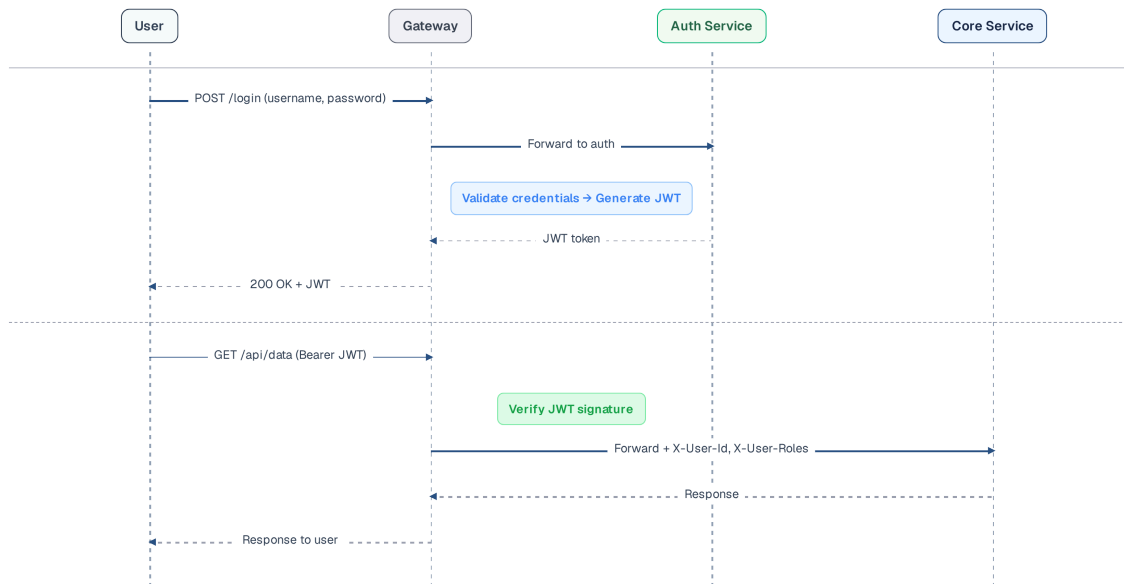
hay **roles**. Thành phần cuối cùng là **Signature**, bảo vệ tính toàn vẹn của token bằng cách băm Header và Payload cùng một khóa bí mật, giúp ngăn chặn việc chỉnh sửa dữ liệu trái phép.

So sánh HS256 và RS256: Hai thuật toán phổ biến này có một số điểm khác biệt quan trọng trong việc thiết lập và xác minh chữ ký:

	HS256 (Symmetric)	RS256 (Asymmetric)
Key	Khóa bí mật dùng chung (Shared secret key)	Khóa riêng tư (ký) + Khóa công khai (xác minh)
Thực hiện ký?	Dịch vụ Xác thực (có khóa bí mật)	Dịch vụ Xác thực (có khóa riêng tư)
Thực hiện xác minh?	Bất kỳ dịch vụ nào có khóa bí mật	Bất kỳ dịch vụ nào có khóa công khai
Security	Nếu secret bị lộ, cả quá trình ký và xác thực đều bị xâm phạm	Nếu private key bị lộ, chỉ quá trình ký bị ảnh hưởng, xác thực vẫn an toàn
Key rotation	Phải update tất cả services	Chỉ update Auth Service (private key)
Use case	Internal services, trust boundary rõ	Public APIs, nhiều third-party verifiers

Bảng 9.2: HS256 (symmetric) vs RS256 (asymmetric)

9.2.3. JWT Flow trong KBM



Hình 9.4: JWT Flow trong KBM — login, token generation, và subsequent requests

i KBM dùng HS256 với shared secret

KBM sử dụng HS256 (symmetric) — các dịch vụ chia sẻ cùng `jwt.secretKey`. Đây là rủi ro đáng kể: nếu bất kỳ dịch vụ nào bị xâm nhập, kẻ tấn công có thể tạo lập chuỗi JWT giả mạo cho **bất kỳ người dùng nào trên toàn hệ thống** (không giới hạn ở dịch vụ bị tấn công). Với KBM (academic, internal network), mức độ rủi ro thấp hơn môi trường mạng diện rộng (public) — nhưng đây *không phải* mô hình bảo mật chấp nhận được cho môi trường thực tế (production). **Lộ trình chuyển đổi** (ưu tiên cao — nhưng vì cần hạ tầng quản lý khóa/JWKS nên được xếp vào *Giai đoạn 2 — Tăng cường Xác thực* ở lộ trình cuối chương, thay vì cải tiến nhanh): (1) chuyển sang RS256 — Dịch vụ Xác thực giữ khóa riêng tư (private key), các dịch vụ khác chỉ sử dụng khóa công khai để xác thực, (2) dùng Spring Security OAuth2 Resource Server (cơ chế xác minh JWT tích hợp sẵn), (3) luân phiên khóa qua config server.

Khi chuyển đổi sang việc luân phiên khóa (key rotation), các rào cản kỹ thuật sẽ bắt đầu bộc lộ.

i Cửa sổ Dual-secret khi Rotate HS256

Luân phiên (rotate) khóa bí mật của HS256 không thể thực hiện tức thời: nếu Dịch vụ Xác thực chuyển sang khóa bí mật mới trong khi các dịch vụ khác vẫn giữ khóa bí mật cũ, mọi token đang lưu hành sẽ ngay lập tức bị xác minh thất bại. Cách an toàn là duy trì một **cửa sổ song mã (dual-secret)**: trong giai đoạn chuyển tiếp, khâu *xác minh* của các dịch vụ sẽ chấp nhận **cả khóa cũ lẫn khóa mới** (`current` + `previous`), còn Dịch vụ Xác thực sẽ ký bằng khóa mới; chỉ gỡ bỏ khóa cũ sau khi mọi token được ký bằng nó đã hoàn toàn hết hạn. Sự phức tạp này là một lý do nữa để chuyển sang cơ chế RS256/JWKS — khi đó chỉ cần công bố thêm khóa công khai mới qua endpoint JWKS, không cần phải đồng bộ khóa bí mật qua tất cả các dịch vụ.

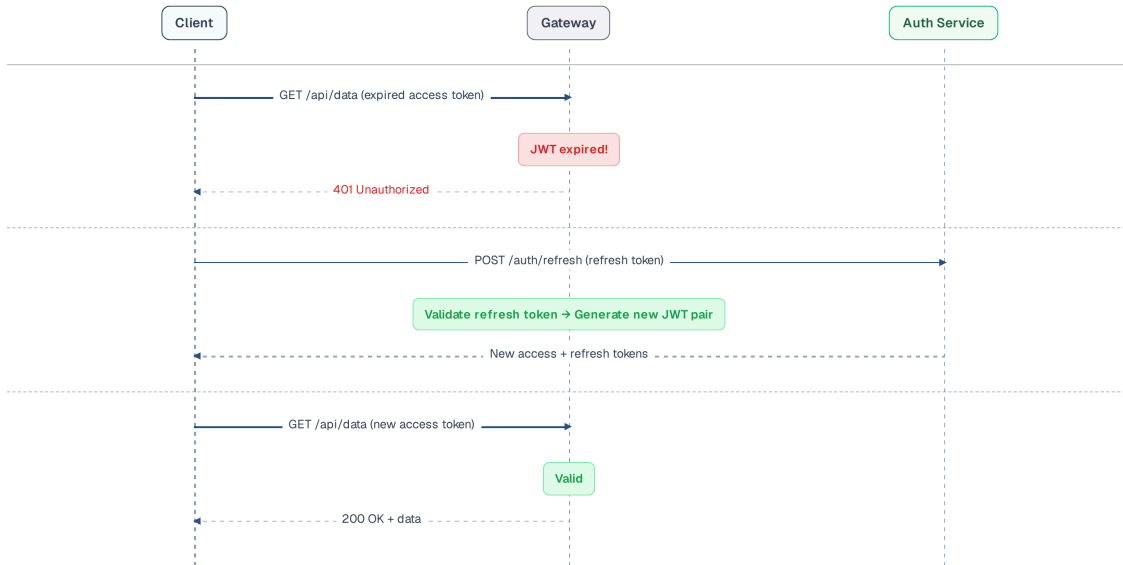
Ẩn dụ: Đăng ký vé dùng máy tính Thư viện

Access Token — là **Vé sử dụng máy tính thư viện 1 giờ** (ngắn hạn để nếu lộ thông tin vé, kẻ gian chỉ sử dụng được 1 giờ).

Refresh Token — là **Thẻ Sinh Viên gốc**, dùng để xuất trình cho Thủ thư cấp vé 1 giờ mới khi hết hạn.

Token Rotation — là việc Thủ thư thu phiếu cũ và cấp phiếu phiên làm việc/tem gia hạn mới mỗi lần xin vé. Nếu kẻ gian nhặt được phiếu cũ và cố xin vé, hệ thống sẽ phát hiện phiếu đã hết hạn và **thu hồi vĩnh viễn** tài khoản đó.

JWT có thời hạn (`exp`). Khi access token hết hạn, người dùng phải **xác thực lại (re-authenticate)** — làm giảm trải nghiệm người dùng đối với các phiên làm việc kéo dài (ví dụ: giảng viên sử dụng hệ thống liên tục suốt một buổi giảng). **Refresh token** giải quyết bài toán này: Token truy cập (Access token) có thời hạn ngắn (15-60 phút), trong khi token làm mới (Refresh token) có thời hạn dài hơn (7-30 ngày). Khi access token hết hạn, client dùng refresh token để nhận access token mới — không cần login lại.



Hình 9.5: Token Refresh & Rotation flow — access token mới + refresh token mới

Token Rotation là chiến lược luân phiên quan trọng: mỗi lần sử dụng token làm mới, **bản thân token đó cũng được thay mới** và token cũ bị vô hiệu hóa. Lợi ích của cơ chế này là: nếu token làm mới bị đánh cắp và sử dụng lại trái phép, Dịch vụ Xác thực (Auth Service) sẽ phát hiện trạng thái đã qua sử dụng của token. Khi đó, hệ thống sẽ tự động vô hiệu hóa toàn bộ chuỗi token liên quan và buộc người dùng phải đăng nhập lại từ đầu [4a, Ch.9].

```

// Auth Service — Token refresh với rotation
@PostMapping("/refresh")
public TokenResponse refreshToken(@RequestBody RefreshRequest request) {
    RefreshToken stored = refreshTokenRepository
        .findByToken(request.getRefreshToken())
        .orElseThrow(() -> new AuthException("Invalid refresh token"));

    // Kiểm tra hết hạn
    if (stored.isExpired()) {
        throw new AuthException("Refresh token expired");
    }

    // Cập nhật atomic
    int updatedRows = refreshTokenRepository.markAsUsedAtomic(stored.getId());
    if (updatedRows == 0) {
        // Phát hiện việc sử dụng lại token! Tiến hành vô hiệu hóa toàn bộ chuỗi token
        refreshTokenRepository.invalidateFamily(stored.getFamily());
        throw new AuthException("Token reuse detected — please login again");
    }

    // Khởi tạo các mã thông báo (token) mới thuộc cùng một chuỗi (family)
    User user = stored.getUser();
    String newAccessToken = jwtUtil.generateAccessToken(user); // 30 phút
    String newRefreshToken = createRefreshToken(user, stored.getFamily()); // 7 ngày

    return new TokenResponse(newAccessToken, newRefreshToken);
}
  
```

Listing 9.1: Token refresh endpoint với rotation

Chiến lược này yêu cầu thiết lập độ dài thời gian sống (TTL) khác biệt cho từng loại token:

Token	Thời hạn	Lưu ở đâu	Lý do
Access token	15-60 phút	Client memory/cookie	Thời gian sống ngắn giúp giảm thiểu rủi ro nếu token bị rò rỉ
Refresh token	7-30 ngày	HttpOnly cookie + DB	Thời gian sống dài duy trì trải nghiệm liền mạch, lưu ở DB để có thể thu hồi
Token family	Theo refresh token	DB (family ID)	Giúp phát hiện việc tái sử dụng trái phép và thu hồi toàn bộ chuỗi

Bảng 9.3: Access token, refresh token và token family — thời hạn, nơi lưu trữ và lý do thiết kế

9.3. Chiến lược Xác thực Kép

Thiết lập vòng bảo vệ danh tính ở lớp ngoài cùng (Gateway) hay tại từng dịch vụ (Service) là một bài toán đánh đổi giữa hiệu năng và độ an toàn (Defense in Depth).

🔍 Trạm kiểm soát cổng trường và Phòng Đào Tạo

Xác thực bề mặt tại Gateway (Claims-only) – Giống như nhân viên an ninh ở trạm kiểm soát cổng chính, chỉ cần nhìn lướt qua thẻ sinh viên xem có dán tem năm nay và có mộc đỏ hay không là cho vào. Nhanh, không nghẽn mạng.

Xác thực sâu tại Auth Service (DB-based) – Giống như khi sinh viên vào Phòng Đào Tạo để xin bằng điểm, chuyên viên phải gõ mã SV vào hệ thống để tra cứu xem sinh viên có đang bị đình chỉ học hay nợ học phí không. Chậm hơn nhưng kiểm tra được trạng thái realtime.

9.3.1. Vấn đề: thực hiện xác thực ở đâu? Mỗi dịch vụ hay chỉ tại gateway?

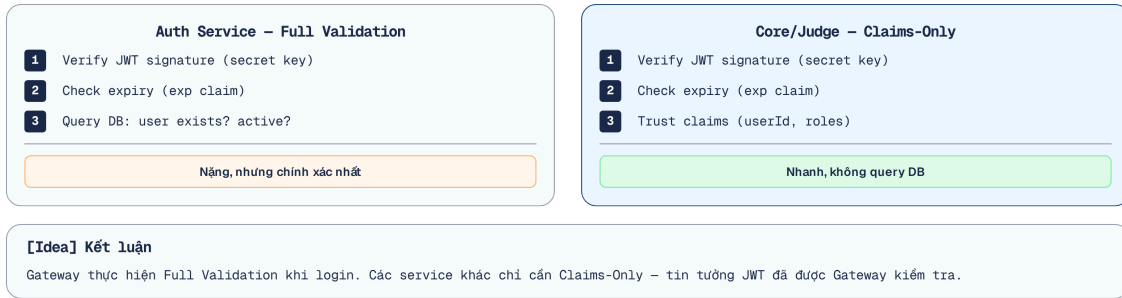
Hai cách tiếp cận trái ngược:

Xác thực đầy đủ tại mỗi dịch vụ — Mỗi dịch vụ sẽ tự xác minh JWT, tự truy vấn cơ sở dữ liệu để kiểm tra xem người dùng còn hoạt động hay không và token chưa bị thu hồi. Ưu điểm: tính bảo mật cao (áp dụng zero trust). Nhược điểm: mỗi dịch vụ đều phải tích hợp thư viện JWT, kết nối cơ sở dữ liệu và bị lặp lại logic xử lý (duplicate logic).

Chỉ xác thực tại Gateway — Chỉ cổng giao tiếp (gateway) thực hiện xác thực, các dịch vụ bên trong tin tưởng hoàn toàn vào các headers được truyền từ gateway. Ưu điểm: kiến trúc đơn giản, tránh việc lặp lại logic. Nhược điểm: nếu kẻ tấn công có thể đi vòng qua gateway (ví dụ: do cấu hình mạng sai lệch), các dịch vụ sẽ hoàn toàn không có lớp bảo vệ nào.

9.3.2. Cách tiếp cận của LMS: Xác thực Kép

LMS triển khai giải pháp **lai (hybrid)** — phương pháp xác thực khác nhau tùy thuộc vào dịch vụ:



Hình 9.6: Xác thực Kép — xác thực đầy đủ (Auth) vs xác thực qua claims (internal services)

Cách tiếp cận này phân chia các mức độ xác thực một cách chi tiết tùy theo ngữ cảnh của dịch vụ, như được tổng hợp dưới đây.

Loại xác thực	Dịch vụ	Khi nào	Chi tiết
Full (DB-based)	Auth Service	Đăng nhập, làm mới token, thao tác nhạy cảm	Xác minh chữ ký + kiểm tra CSDL (tài khoản còn hiệu lực, token hợp lệ)
Claims-only	Gateway	Mọi yêu cầu	Xác minh chữ ký JWT + thời gian hết hạn (stateless)
Header-trust	Core, Judge, Assignment	Mọi request (sau gateway)	Tin tưởng <code>X-User-Id</code> và <code>X-User-Roles</code> từ gateway

Bảng 9.4: Ba loại xác thực trong KBM

Auth Service (Xác thực đầy đủ): xác minh chữ ký và thời hạn JWT, truy vấn CSDL (“người dùng còn tồn tại và đang hoạt động?”), kiểm tra danh sách đen của token. Quá trình này tiêu tốn nhiều tài nguyên xử lý nhưng đảm bảo tính chính xác tuyệt đối — được áp dụng cho các luồng đăng nhập, làm mới token và các thao tác nhạy cảm.

Core/Judge Service (claims-only): nhận `X-User-Id` và `X-User-Roles` từ headers (do gateway chèn vào). Không cần xác thực JWT lại — chỉ tập trung kiểm tra quyền hạn: `if (!roles.contains("STUDENT")) throw ForbiddenException`. Xử lý nhanh, hoạt động phi trạng thái (stateless) và hoàn toàn tin cậy vào kết quả xác thực từ gateway (do lưu lượng mạng thuộc nội vùng).

Mô hình này được gọi là **lan truyền định danh dựa trên claims**: cổng giao tiếp (gateway) chịu trách nhiệm kiểm chứng token, các dịch vụ phía sau tin tưởng hoàn toàn vào gateway và chỉ tập trung vào việc phân quyền (authorization).

⚠ Bảo mật Header và Truyền định danh (Identity Propagation)

Việc Core Service hoàn toàn tin tưởng header `X-User-Id` do Gateway truyền xuống cũng giống như giám thị phòng thi tin tưởng chiếc thẻ sinh viên đã được ban tổ chức kiểm tra kỹ lưỡng ở cổng. Sẽ rất nguy hiểm nếu sinh viên tự in thẻ giả (cố tình nhét header `X-User-Id: admin` từ trình duyệt). Do đó, Gateway bắt buộc phải có luật **gỡ bỏ hoàn toàn (tẩy trắng)** các header `X-User-*` đến từ Internet trước khi tiến hành xác thực và tự động chèn lại thẻ thật.

Đồng thời, đối với các giao tiếp nội bộ (ví dụ: Job Scheduler gọi trực tiếp Core Service) mà đi cửa sau bỏ qua Gateway, service gọi sẽ không có “thẻ” do Gateway cấp. Để khắc phục, hệ thống phải dùng cơ chế **Client Credentials** như một loại giấy đi đường đặc biệt cho giao tiếp Machine-to-Machine (M2M), hoặc Service đích phải tự bổ sung một Filter giải mã và xác thực JWT cho các request đi theo đường tắt.

🔐 Xác thực tại biên, Phân quyền tại dịch vụ

Gateway chịu trách nhiệm *authentication* (ai đang gọi?). Service chịu trách nhiệm *authorization* (người này có quyền làm hành động này?). Việc tách biệt xác thực và phân quyền cho phép mỗi lớp tập trung vào một nhiệm vụ chuyên biệt: cổng giao tiếp không cần nắm giữ các quy tắc nghiệp vụ (business rules), và dịch vụ không cần quan tâm đến định dạng JWT.

9.4. OAuth2 – External Identity Provider

Ủy quyền xác thực (Delegated Authentication) cho các nhà cung cấp định danh bên ngoài giúp giảm thiểu rủi ro bảo mật từ việc tự quản lý mật khẩu, đồng thời mang lại trải nghiệm SSO liền mạch cho người dùng.

Vấn đề quản lý credentials phức tạp: Việc tự quản lý thông tin đăng nhập đòi hỏi phải xử lý nhiều nghiệp vụ phức tạp: băm mật khẩu (hash passwords), cấp lại mật khẩu đã quên, xác thực đa yếu tố (2FA), chống tấn công dò mật khẩu (brute force protection) và tuân thủ các quy chuẩn bảo mật. Với đội ngũ nhỏ, **ủy quyền xác thực (delegate authentication)** cho identity provider chuyên nghiệp (Google, Microsoft, GitHub) giúp giảm đáng kể công sức và rủi ro.

9.4.1. OAuth2 Authorization Code Flow

💡 Mua Laptop giảm giá cho sinh viên

Hãy tưởng tượng một sinh viên ra FPT Shop mua laptop giảm giá sinh viên. FPT Shop (Client) không được phép hỏi mật khẩu Cổng thông tin đào tạo của sinh viên (rất nguy hiểm). Thay vào đó, FPT chuyển hướng người dùng về trang web của trường để đăng nhập. Sau khi đăng nhập, trường cấp cho người dùng một “Vé chứng nhận sinh viên” (Authorization Code) dùng 1 lần. Hệ thống backend của FPT mang vé đó đến hệ thống của trường để đổi lấy “Voucher giảm giá” (Token).

KBM tích hợp OAuth2/OIDC để thực hiện đăng nhập thông qua nhà cung cấp danh tính của trường (ví dụ: Microsoft Entra ID hoặc Google Workspace). Quy trình hoạt động như sau: Người dùng được chuyển hướng tới hệ thống cung cấp danh tính để đăng nhập. Khi thành công, hệ thống nhận về mã ủy quyền (authorization code) thông qua một lệnh gọi lại (callback). Tiếp đó, Dịch vụ Xác thực sử dụng mã này để đổi lấy token thông qua giao tiếp trực tiếp giữa các máy chủ (server-to-server). Cuối cùng, dịch vụ truy xuất thông tin người dùng, tìm kiếm hoặc tạo mới tài khoản tương ứng và phát hành chuỗi KBM JWT nội bộ.

Điểm quan trọng: KBM *không sử dụng* trực tiếp token của bên thứ ba. Sau khi xác thực thành công, Dịch vụ Xác thực sẽ tạo ra một **JWT nội bộ riêng của KBM** — các dịch vụ xử lý phía sau hoàn toàn không cần biết người dùng đã đăng nhập bằng Microsoft, Google, hệ thống Quản lý Đào tạo hay qua mật khẩu truyền thống. Mô hình này được gọi là hoán đổi token (token exchange): chuyển đổi từ external token sang internal token.

9.4.2. Multiple Authentication Methods

KBM hỗ trợ nhiều phương thức xác thực nhưng hội tụ về cùng một token nội bộ:

Method	Flow	Khi nào
Username/Password	Truyền thống, hash bcrypt	Default cho mọi user
Microsoft/Google OAuth2/OIDC	Authorization Code Flow + PKCE (kèm <code>state</code> và <code>nonce</code> chống CSRF) khi phù hợp	SSO bằng tài khoản trường
Cổng QLĐT (Quản lý Đào tạo)	Custom integration	Login bằng tài khoản quản lý đào tạo
Xác thực email	Xác thực email trước khi kích hoạt/khôi phục	Giảm tài khoản giả, hỗ trợ cấp lại mật khẩu
Turnstile	Challenge chống bot ở form nhạy cảm	Bảo vệ login/register/reset khỏi automation

Bảng 9.5: Các phương thức xác thực trong KBM

Tất cả các phương thức này đều hội tụ về một đầu ra duy nhất: **KBM JWT token** (chứa `userId`, `roles`, và `expiry`). Dịch vụ Xác thực cung cấp hàm `generateTokens(user)` — tiếp nhận thực thể Người dùng từ bất kỳ phương thức đăng nhập nào và trả về cặp thẻ `{accessToken, refreshToken}`. Các dịch vụ ở tuyến sau (downstream services) không cần phân biệt nguồn gốc xác thực. Đây là nền tảng cho cơ chế Đăng nhập một lần (SSO) đa nền tảng: các module LMS core, Network Lab và DevOps Lab có thể chia sẻ identity nội bộ mà không buộc từng module hiểu mọi provider bên ngoài.

9.4.3. Service-to-Service: Client Credentials Flow

Lưu ý: Luồng Authorization Code chỉ dành cho tương tác có mặt người dùng (User-facing). Khi một service tự động gọi một service khác ở chế độ background (Machine-to-Machine, ví dụ: Job Scheduler gọi Judge Service), ta phải sử dụng **Client Credentials Flow**. Lúc này, service gọi hoạt động như một client có `clientId` và `clientSecret` riêng để tự xin access token từ Auth Server mà không cần người dùng đăng nhập.

9.5. RBAC — Role-Based Access Control

Phân quyền giống như việc cấp thẻ từ cho các khu vực khác nhau trong trường: thẻ của sinh viên chỉ mở được cửa thư viện và giảng đường, trong khi thẻ của giảng viên có thể mở thêm được cửa phòng thí nghiệm chuyên sâu. Việc quản lý truy cập theo vai trò giúp hệ thống dễ dàng kiểm soát ai được phép làm gì.

🔗 Authentication vs Authorization

Xác thực (Authentication - AuthN) – Trình thẻ để chứng minh “Tôi là sinh viên Nguyễn Văn A”.

Phân quyền (Authorization - AuthZ) – Ổ khóa điện tử kiểm tra xem thẻ của Nguyễn Văn A có chữ **GIANG_VIEN** để được phép mở cửa Phòng nghỉ Giảng viên hay không.

9.5.1. Vấn đề: ai được làm gì?

Authentication trả lời “người dùng là ai?”. Authorization trả lời “người dùng được làm gì?”. Trong KBM, ba vai trò chính cần permissions khác nhau:

Tại KBM, hệ thống phân quyền được xây dựng trên cơ sở ba cấp bậc rõ ràng. Ở cấp độ thấp nhất, **STUDENT** (Sinh viên) chỉ được cấp các quyền cơ bản như nộp bài, xem kết quả và tham gia kỳ thi. Vai trò **LECTURER** (Giảng viên) kế thừa toàn bộ quyền của sinh viên, đồng thời được trao thêm khả năng tạo lập bài tập, khởi tạo kỳ thi và xem các báo cáo thống kê. Cuối cùng, **ADMIN** có quyền cao nhất, bao gồm toàn bộ quyền của giảng viên và quyền quản lý tài khoản, cấu hình hệ thống.

9.5.2. RBAC Implementation trong KBM

KBM dùng Spring Security `@PreAuthorize` annotation với roles lưu trong JWT:

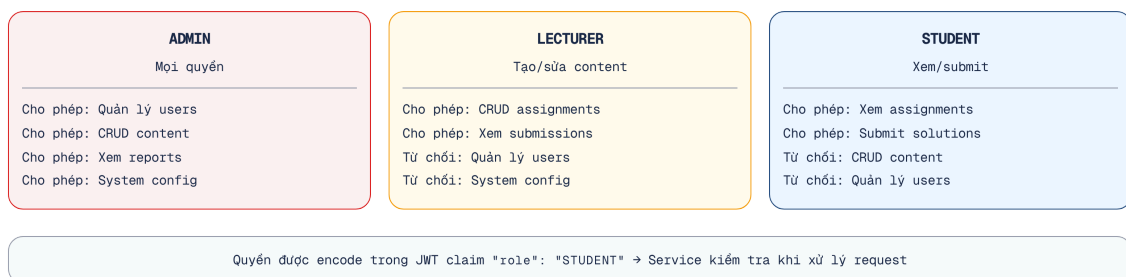
```
// Spring Security @PreAuthorize — declarative RBAC
@PreAuthorize("isAuthenticated()")
@GetMapping("/questions")
public List<QuestionResponse> getQuestions() { ... } // Mọi user authenticated

@PreAuthorize("hasAnyRole('LECTURER', 'ADMIN')")
@PostMapping("/questions")
public QuestionResponse create(@RequestBody CreateQuestionRequest r) { ... }

@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/questions/{id}")
public void delete(@PathVariable UUID id) { ... }
```

Listing 9.2: Spring Security `@PreAuthorize` — RBAC declarative

Role Hierarchy: Sơ đồ dưới đây minh họa sự kế thừa quyền hạn giữa các vai trò:



Hình 9.7: Role Hierarchy — ADMIN inherits LECTURER inherits STUDENT

Để áp dụng hệ thống phân cấp này vào mã nguồn, ta có thể cấu hình trực tiếp thông qua Spring Security.

i Actionable Code: RoleHierarchy & OAuth2

Để cấu hình **RoleHierarchy** trong Spring Security (tránh việc phải kiểm tra cứng nhiều vai trò), chúng ta cần khai báo một **@Bean** :

```
@Bean
public RoleHierarchy roleHierarchy() {
    RoleHierarchyImpl r = new RoleHierarchyImpl();
    r.setHierarchy("ROLE_ADMIN > ROLE_LLECTURER \n ROLE_LLECTURER > ROLE_STUDENT");
    return r;
}
```

Chi tiết về cấu hình OAuth2 Client với Google/Entra ID được cung cấp đầy đủ trong *source code Companion Repo*.

KBM lưu roles dạng **pipe-delimited** trong JWT: **"ADMIN|LECTURER|STUDENT"** . Khi gateway trích xuất các vai trò, nó truyền qua header **X-User-Roles** — Core Service sau đó sẽ phân tích chuỗi và kiểm tra quyền truy cập thực tế.

9.5.3. API-driven Route Protection (Frontend)

KBM frontend sử dụng pattern khác biệt: **route permissions** được lấy từ API (**GET /api/auth/me/permissions**) thay vì hardcoded — response chứa danh sách routes và actions mà người dùng được phép. Giao diện (frontend) sẽ thực hiện hiển thị động (dynamic render) các tuyến đường (routes) dựa trên những phân quyền này. Ưu điểm lớn nhất là khi có thay đổi về phân quyền ở phía máy chủ (backend), giao diện máy khách sẽ tự động cập nhật cấu trúc mà không cần phải trải qua quá trình triển khai (deploy) lại.

i RBAC trong KBM

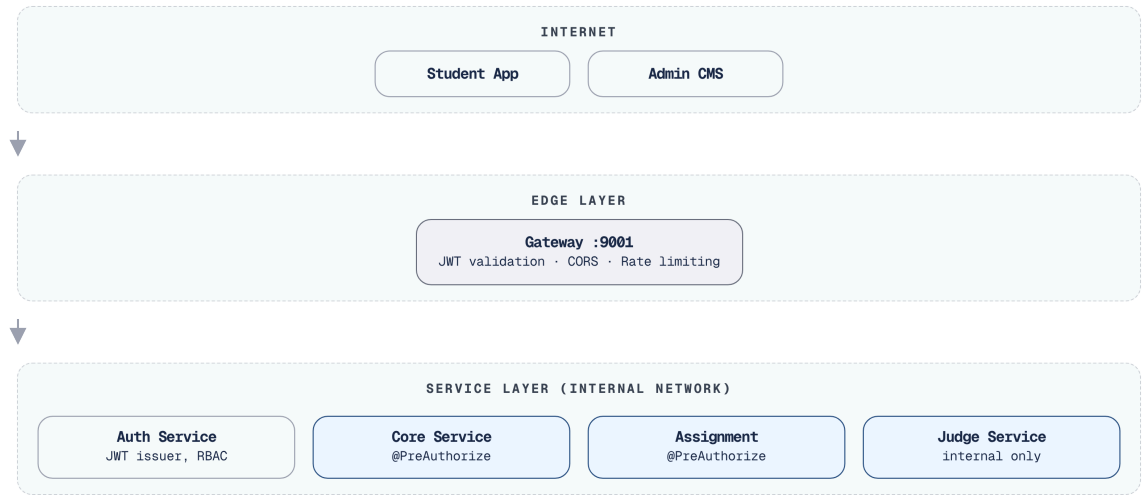
Cách thức triển khai của KBM hiện tại còn một số hạn chế: (1) Các vai trò (roles) được lưu dưới dạng chuỗi phân tách bằng dấu gạch đứng (pipe-delimited string) thay vì cấu trúc mảng. Điều này bắt buộc hệ thống phải phân tích cú pháp một cách thủ công, dễ phát sinh lỗi nếu tên vai trò có chứa ký tự phân cách. (2) Các chú thích **@PreAuthorize** nằm rải rác ở từng controller — gây khó khăn cho quá trình kiểm toán (audit) quyền hạn của một vai trò cụ thể trên toàn bộ các endpoint. (3) Hệ thống thiếu một hệ thống phân cấp vai trò (role hierarchy) rõ ràng ở tầng Spring Security — dẫn đến việc vai trò ADMIN phải liệt kê lại toàn bộ các quyền của LECTURER và STUDENT. (4) Phân hệ DevOps Lab cần bổ sung các chính sách kiểm soát tại tầng thực thi (runtime layer) để xác định rõ người dùng nào được phép khởi tạo không gian làm việc, vùng mạng cách ly, container hoặc các công việc đòi hỏi đặc quyền cao.

Lộ trình chuyển đổi (Migration path): (1) chuyển đổi cấu trúc lưu trữ vai trò sang mảng JSON (JSON array) bên trong JWT payload, (2) thiết lập cấu hình **RoleHierarchy** thống nhất trong Spring Security, và (3) cân nhắc việc quy tụ các chính sách phân quyền về một nền tảng quản lý tập trung (như Spring Security method security hoặc Open Policy Agent).

9.6. Case Study: KBM

Phần này tổng hợp đánh giá tổng thể kiến trúc bảo mật của KBM, nhận diện các lỗ hổng tiềm ẩn (gap analysis) và đề xuất lộ trình khắc phục (migration roadmap) theo từng giai đoạn.

Tổng quan



Hình 9.8: Kiến trúc bảo mật tổng thể của KBM

9.6.1. Phân tích tổng hợp

Việc đánh giá chi tiết kiến trúc bảo mật của KBM giúp nhận diện những lỗ hổng tiềm ẩn (tương tự như công tác thanh tra giáo dục đối với quy trình tổ chức thi). Bảng dưới đây tổng hợp các khía cạnh bảo mật, rủi ro tương ứng và đề xuất biện pháp phòng ngừa:

Aspect	Hiện trạng	Risk Level	Best Practice
JWT Algorithm	HS256 (symmetric, shared secret)	! Medium	RS256 cho production
Token Storage	Client-side (localStorage)	! Medium	HttpOnly cookie (XSS protection)
Secret Management	Một phần hardcoded trong application.yml, một phần qua environment variables	● High	Vault, AWS Secrets Manager
CORS	allowedOrigins: ""	● High	Restrict to specific origins
Service-to-service auth	Không có	! Medium	mTLS hoặc service token
Rate Limiting	Không có	! Medium	Redis-based rate limiter
Xác thực email / Chống bot	Có/đăng hoàn thiện theo module	! Medium	Xác thực email + Turnstile cho form nhạy cảm
DevOps Lab isolation	Runtime isolation riêng	● High nếu cấu hình sai	Sysbox/k3s + NetworkPolicy/RBAC
Xác thực dữ liệu đầu vào	Một phần	! Medium	Sử dụng Bean Validation (@Valid)
HTTPS	Gateway level	✓ Low	—
Password Hashing	BCrypt	✓ Low	—

Bảng 9.6: Phân tích bảo mật KBM

Trong quá trình đánh giá các thuật toán đang sử dụng, đặc biệt lưu ý đến các tiêu chuẩn mã hóa bắt buộc.

⚠ Độ dài khóa HS256

Với thuật toán HS256, giá trị của `jwt.secretKey` bắt buộc phải là một chuỗi ngẫu nhiên có độ dài tối thiểu 256 bits (32 bytes). Các thư viện JWT hiện đại (như `jjwt` bản mới) sẽ throw `WeakKeyException` lúc khởi động nếu khóa quá ngắn.

9.6.2. Đề xuất migration

Giai đoạn 1 — Các cải tiến nhanh (Quick Wins) (giải pháp ngắn hạn tối ưu: nỗ lực thực hiện thấp nhưng mang lại hiệu quả cao):

- Giới hạn nguồn gốc CORS (Restrict CORS origins) (Lưu ý: Nếu kích hoạt HttpOnly cookie (kéo theo `allowCredentials=true`), hệ thống tuyệt đối không được sử dụng `allowedOrigins("")` nhằm tránh phát sinh lỗi tương thích).
- Di chuyển các thông tin nhạy cảm (secrets) ra khỏi application.yml → environment variables (tối thiểu).
- Lưu trữ token trong HttpOnly cookie thay vì localStorage.

- Kích hoạt xác thực email và Turnstile cho login/register/reset ở các luồng giao tiếp công khai (public flows).

Giai đoạn 2 — Tăng cường Xác thực (Authentication Hardening) (nỗ lực triển khai trung bình):

- Chuyển đổi thuật toán mã hóa từ HS256 sang RS256
- Thống nhất phiên bản thư viện JWT trên toàn hệ thống (như đã đề cập ở Chương 8)
- Triển khai luồng làm mới token kết hợp luân phiên khóa (mỗi lần làm mới sẽ vô hiệu hóa token cũ)
- Chuẩn hóa cơ chế đăng nhập một lần (SSO) đa nền tảng theo mô hình: hoán đổi token bên ngoài thành token nội bộ KBM, dựa trên một hợp đồng dữ liệu chung (shared claims contract)

Giai đoạn 3 — Bảo mật Service Mesh (nỗ lực triển khai cao, áp dụng khi mở rộng quy mô):

- Service-to-service authentication (mTLS hoặc internal JWT)
- Centralized secret management (HashiCorp Vault)
- API-level authorization policies (Open Policy Agent)
- DevOps Lab runtime guardrails: Kubernetes RBAC, NetworkPolicy, resource quotas và Sysbox isolation

9.6.3. Secret Management trong Production

9.6 cho thấy “Secret Management” là risk level ● High — `jwt.secretKey` hardcoded trong `application.yml` và commit vào Git. Đây là lỗi hỏng phổ biến nhất trong microservices và cần được xử lý ở mọi hệ thống production.

Ba cấp độ Secret Management:

Cấp độ	Cách tiếp cận	Ưu điểm	Nhược điểm	Phù hợp
Level 1	Environment Variables	Đơn giản, tách secret khỏi code	Secrets trong plaintext trên server, không audit trail	Dev/Staging
Level 2	Encrypted Config (Spring Cloud Config + Jasypt)	Secrets mã hóa trong repo, decrypt lúc runtime	Vẫn cần quản lý encryption key	Small production
Level 3	Secret Manager (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager)	Auto-rotation, audit log, access policies, dynamic secrets	Infrastructure cost, operational complexity	Production at scale

Bảng 9.7: Chiến lược Secret Management — từ cơ bản đến enterprise

Level 1 — Environment Variables (LMS nên áp dụng ngay):

Thay vì:

```
# sym.times application.yml — KHÔNG BAO GIỜ commit secrets
jwt:
  secretKey: "mySecretKey123"
  expiration: 86400000
```

Sử dụng:

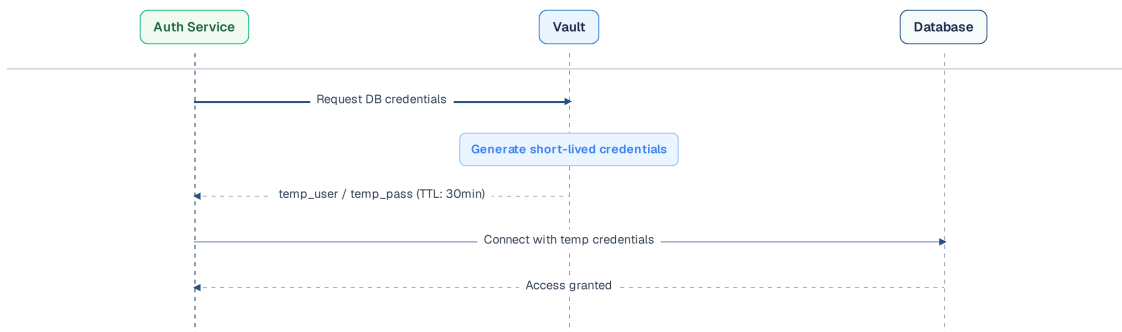
```
# sym.checkmark application.yml — reference environment variable
jwt:
```

```
secretKey: ${JWT_SECRET_KEY}
expiration: ${JWT_EXPIRATION:86400000}
```

```
# Docker Compose — inject từ .env file (không commit .env)
services:
  kbm-auth:
    environment:
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
      - DB_PASSWORD=${DB_PASSWORD}
```

Level 3 — HashiCorp Vault (khi hệ thống production hoàn chỉnh):

Vault cung cấp **dynamic secrets** — database credentials được tạo tự động, có thời hạn, và auto-rotate. Không ai biết password database thực sự — kể cả developer.



Lợi ích

Credentials tự hết hạn. Không hardcode password. Vault tự rotate. Audit trail cho mọi access.

Hình 9.9: Vault Dynamic Secrets — credentials tạm thời, tự động xoay vòng

💡 .env file và .gitignore

Bước đầu tiên và quan trọng nhất: thêm `.env` vào `.gitignore`. Tạo `.env.example` (không chứa các giá trị thực) để các thành viên trong nhóm phát triển nắm được những biến cần thiết lập. Đây là một ‘cải tiến bảo mật không tốn chi phí’ (zero-cost security improvement) mà mọi dự án đều nên áp dụng ngay lập tức.

Một số lỗ hổng bảo mật thường gặp ở mức mã nguồn và cấu hình dịch vụ:

⚠ Mã nguồn và cấu hình**Lưu JWT trong localStorage**

Bất kỳ đoạn mã JavaScript nào chạy trên trang cũng đều có thể đọc được dữ liệu này (tấn công XSS). Hậu quả: nếu tồn tại lỗ hổng XSS, kẻ tấn công có thể đánh cắp token và từ đó mạo danh người dùng.

Phòng tránh: lưu trữ JWT trong HttpOnly cookie; loại cookie này không thể bị truy xuất bởi JavaScript và trình duyệt sẽ tự động đính kèm trong các yêu cầu gửi đi.

Hardcode secrets trong source code

`jwt.secretKey=mySecretKey123` trong application.yml, commit vào Git. Hậu quả: ai có access repo đều có thể tạo JWT giả mạo. **Phòng tránh:** dùng environment variables (tối thiểu), hoặc secret manager (Vault, AWS Secrets Manager).

Không set expiry cho JWT

Token không bao giờ hết hạn. Hậu quả: nếu token bị rò rỉ, kẻ gian sẽ có quyền truy cập vĩnh viễn và hệ thống không có cách nào để thu hồi. **Phòng tránh:** sử dụng token truy cập ngắn hạn (15-60 phút), kết hợp với token làm mới có thời hạn dài hơn (7-30 ngày) đi kèm cơ chế luân phiên (rotation).

Xác thực JWT ở mỗi dịch vụ bằng cách khác nhau

Mỗi dịch vụ sử dụng một phiên bản thư viện JWT khác nhau và tự phân tích token theo cách riêng. Hậu quả: gateway chấp nhận nhưng dịch vụ lại từ chối (hoặc ngược lại), làm cho quá trình gỡ lỗi trở nên rất khó khăn. **Phòng tránh:** thống nhất việc xác thực tại gateway, các dịch vụ bên trong chỉ tin tưởng headers (§9.3).

Confused Deputy Problem

Service A gọi Service B thay mặt user, nhưng B không biết A đang “đại diện” cho user hay hành động với quyền riêng của A. Newman trong [4a, Ch.9] gọi đây là **confused deputy attack**. Hậu quả: Service A có thể vô tình escalate privileges — user chỉ có quyền STUDENT nhưng Service A gọi B với full service credentials. **Phòng tránh:** luôn truyền user identity (JWT hoặc headers `X-User-Id` , `X-User-Roles`) trong service-to-service calls — B authorize dựa trên **user** chứ không phải **service gọi**.

Mặc dù có nhiều tài liệu giải thích, việc tự trải nghiệm các cuộc tấn công giúp đội ngũ kỹ sư cảnh giác hơn.

🔗 Interactive Demo

Xem minh họa động tại companion web:

Luồng xác thực OAuth2 & JWT – – oauth2-jwt-flow.html

Role-Based Access Control (RBAC) – – rbac-authorization.html

⚠ Sai lầm thường gặp

Dùng HS256 shared secret cho production

Tất cả các dịch vụ chia sẻ chung một khóa bí mật để ký và xác thực JWT. Hậu quả: nếu chỉ một dịch vụ bị xâm nhập, kẻ tấn công có thể tạo token giả mạo cho bất kỳ người dùng nào trên toàn hệ thống.

Phòng tránh: chuyển sang sử dụng thuật toán phi đối xứng RS256/JWKS — theo đó, chỉ duy nhất Dịch vụ Xác thực mới được giữ khóa riêng tư (private key) để ký, còn các dịch vụ khác chỉ sử dụng khóa công khai (public key) để tiến hành xác minh.

Tin tưởng dữ liệu định danh do client gửi

Đọc thẳng header như `X-User-Id` hoặc tin tưởng vào claim chưa được kiểm chứng. Hậu quả: leo thang đặc quyền, confused deputy. **Phòng tránh:** gateway gỡ bỏ các header nhạy cảm từ phía client và chỉ gắn lại sau khi quá trình xác thực hoàn tất; dịch vụ trực tiếp kiểm chứng chữ ký của token, không tin tưởng vào các header chưa được xác thực.

Token sống quá lâu, không thể thu hồi

Access token có thời hạn quá dài, không đi kèm cơ chế làm mới hoặc luân phiên. Hậu quả: token bị lộ vẫn có thể được sử dụng trong thời gian dài mà không thể thu hồi. **Phòng tránh:** kết hợp token truy cập ngắn hạn với cơ chế luân phiên token làm mới, đồng thời giám sát để phát hiện tái sử dụng trái phép (vô hiệu hóa toàn bộ chuỗi token).

Hardcode secret trong source/application.yml

Lưu secret, key, mật khẩu DB trực tiếp trong file cấu hình commit lên Git. Hậu quả: rò rỉ secret qua lịch sử Git. **Phòng tránh:** dùng biến môi trường / secret manager (Vault), không commit secret.

9.7. Cô lập thực thi và Lockdown thi cử — Hai bài học phòng thủ

Bảo mật không dừng ở JWT và RBAC. Hai nghiệp vụ chấm code và thi phòng máy của KBM cho thấy hai mặt khác của phòng thủ: **cô lập tài nguyên** khi phải chạy mã không tin cậy, và **defense-in-depth** khi chống gian lận.

9.7.1. Chạy mã sinh viên: rủi ro “no sandbox” và lời giải cô lập

Bộ chấm code thế hệ đầu (`kbm-judge-code` Gen 1) chạy bài nộp C/C++ và Java **trực tiếp trong tiến trình máy chủ, không có sandbox**. Đây là lỗ hổng nghiêm trọng: mã sinh viên là *dữ liệu đầu vào không đáng tin cậy có khả năng thực thi* — nó có thể đọc/ghi file hệ thống, thiết lập kết nối mạng trái phép ra bên ngoài, tiêu thụ vô hạn CPU/RAM, hoặc cố ý gây ngừng trệ dịch vụ. Khác với chấm SQL (chạy trong sandbox CSDL cô lập mỗi lần submit) hay lab DevOps (pod Sysbox cô lập), judge code Gen 1 bỏ qua hoàn toàn lớp cô lập — một dạng đánh đổi “tự viết để kiểm soát tối đa” nhưng vô tình mở rộng bề mặt tấn công ngay bên trong ranh giới tin cậy của dịch vụ.

⚠ Không bao giờ chạy untrusted code chung tiến trình

Mã do người dùng nộp lên (judge code, plugin, công thức tùy biến...) phải được xem như **thù địch cho đến khi được chứng minh ngược lại**. Chạy nó chung tiến trình/máy chủ với service đồng nghĩa trao cho kẻ tấn công chính quyền hạn của service đó: truy cập file, mạng, secret trong biến môi trường, và tài nguyên không giới hạn. Lỗ hổng này không phải lỗi logic — nó là *thiếu vắng một lớp cô lập*.

Thế hệ hiện tại tích hợp **DOMjudge**, chạy mỗi bài nộp trong **sandbox cô lập** dựa trên cgroup/namespace của Linux: giới hạn CPU, bộ nhớ, thời gian thực thi, và cắt quyền truy cập tài nguyên ngoài. Nguyên tắc bảo

mật cốt lõi: **không bao giờ chạy mã không tin cậy trong cùng ranh giới tin cậy (trust boundary) với dịch vụ**. Ở đây, cô lập (sandbox) đi song hành với quyết định tự xây dựng hay mua sẵn (build-vs-buy) (Chương 3, Chương 4) — chọn một judge OSS đã được kiểm chứng qua hàng nghìn kỳ thi vừa giải bài toán đa ngôn ngữ, vừa đóng lại lỗ hổng cô lập mà bản tự viết để hở.

9.7.2. Lockdown thi cử và defense-in-depth chống gian lận

Với kỳ thi tại phòng máy, KBM dùng `kbm-exam-client` — một desktop client siết môi trường thi: bắt buộc toàn màn hình, chặn Alt-Tab/chuyển ứng dụng, vô hiệu copy-paste và chuột phải, chặn mở trình duyệt/tài liệu ngoài, và chỉ cho kết nối từ dải IP **whitelist** của phòng thi. Client còn thu tín hiệu giám sát (rời cửa sổ, mất focus, cảm/rút thiết bị) và đẩy về server làm dữ liệu anti-cheat. Client này dùng cùng JWT của `kbm-auth` (SSO) và là một client của bounded context Examination & Proctoring.

Tuy nhiên, việc phong tỏa (lockdown) ở phía máy khách **không bao giờ được coi là lớp phòng thủ duy nhất**. Một thí sinh có kỹ thuật hoàn toàn có thể vá, giả lập, hoặc đơn giản là bỏ qua client desktop — mọi thứ chạy trên máy người dùng đều nằm dưới quyền kiểm soát của người dùng. Vì vậy server (logic Contest/Anti-cheat, hiện trong `kbm-core`) phải **độc lập** phát hiện bất thường: đổi IP hoặc User-Agent giữa ca thi, tần suất submit bất thường, số lần vi phạm vượt ngưỡng cấu hình. Đây chính là **defense-in-depth** (§9.1) áp dụng cho anti-cheat: tín hiệu được thu cả ở client (lockdown) lẫn ở server (phát hiện bất thường), không phía nào là điểm tin cậy tuyệt đối.

⚠ Không bao giờ tin tuyệt đối vào client

Lockdown của `kbm-exam-client` làm *tăng chi phí gian lận*, không phải *loại bỏ* khả năng gian lận. Tương tự như nguyên lý khi cổng giao tiếp gỡ bỏ header `X-User-*` từ Internet (§9.3): mọi tín hiệu bảo mật do client gửi lên đều phải được server kiểm chứng hoặc đối chiếu độc lập. Máy khách đảm nhiệm trải nghiệm người dùng và đóng vai trò là lớp ngăn chặn đầu tiên; máy chủ nắm quyền quyết định cuối cùng và là nguồn chân lý cho anti-cheat.

9.8. Tổng kết

Bảo mật microservices phức tạp hơn monolith vì attack surface lớn hơn — mỗi service, mỗi network call, mỗi database đều là điểm tiềm ẩn rủi ro. Defense in depth — bảo vệ nhiều lớp, không dựa vào một lớp duy nhất — là nguyên tắc nền tảng.

JWT giải quyết bài toán authentication trong distributed system: stateless, tự chứa identity information, không cần shared session store. HS256 (symmetric) đơn giản nhưng đòi hỏi shared secret — phù hợp cho internal services. RS256 (asymmetric) an toàn hơn cho production với nhiều verifiers.

Xác thực kép — xác thực đầy đủ tại Dịch vụ Xác thực, dựa trên claims tại cổng giao tiếp, và tin tưởng header tại các dịch vụ nội bộ — là cách tiếp cận thực tế: cân bằng giữa security và performance. OAuth2 cho phép delegate authentication cho identity provider chuyên nghiệp, giảm effort quản lý credentials.

RBAC là mô hình authorization phổ biến nhất — đủ tốt cho hầu hết hệ thống. KBM implement RBAC qua `@PreAuthorize` annotations, nhưng thiếu role hierarchy rõ ràng và authorization policy tập trung.

Phân tích KBM cho thấy kiến trúc bảo mật cơ bản đúng (JWT, xác thực tại gateway, RBAC, token exchange cho SSO) nhưng có nhiều quick wins cần xử lý: CORS restriction, secret management, token storage, xác thực email/Turnstile và runtime isolation cho DevOps Lab. Đây là technical debt bảo mật — không ảnh hưởng tính năng nhưng ảnh hưởng risk profile.

Ở Chương 10, chúng ta sẽ tổng hợp mọi kiến thức từ Chương 1-9 vào **bài toán thực tế nhất**: chuyển đổi từ monolith sang microservices — khi nào nên chuyển, Strangler Fig pattern, tách database, và migration roadmap cho KBM.

Bài tập Chương 9

Xem Phụ lục Bài tập để luyện tập:

9-1 – Security Incident: JWT Token Theft (Debugging [L3])

9-2 – OAuth2 + OIDC Flows — Chọn flow nào cho scenario nào? (Trade-off Analysis [L2])

9.9. Đọc thêm

Sách tham khảo chính:

1. [4a] Sam Newman, *Building Microservices* — Ch.9: Security
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.11: Developing secure services, Access Token pattern
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution (data formats, schema evolution — nền tảng cho hiểu binary/JSON encoding trong tokens và API messages)

Nguồn trực tuyến:

- JWT.io — jwt.io (interactive JWT decoder)
- OWASP API Security Top 10 — owasp.org/www-project-api-security
- Spring Security OAuth2 Resource Server — docs.spring.io/spring-security
- RFC 7519 — JSON Web Token (JWT) specification

Tài liệu tham khảo

- Newman, S. (2021). *Building Microservices*, Chapter 11. O'Reilly Media.